PRODUCT CASE STUDY

Improving Feature Adoption in Enterprise SaaS

A structured approach to diagnosing low adoption, understanding users, and shipping the right fixes.

Vagisha Anand · Product & Business Analyst

PROBLEM STATEMENT

BrowserStack Built It. Nobody Came.

BrowserStack's Live and App Live products offer QA engineers a native integration to log bugs directly into JIRA - without leaving the testing console. It's a meaningful workflow shortcut for teams that spend their days context-switching between tools.

But months after launch, adoption is stubbornly low - and it's not isolated. It's low across all user segments, all regions, all product versions.

Why this matters beyond feature usage

→ Integrations are BrowserStack's stickiness lever — when QA teams log bugs from inside the platform, it becomes embedded in their daily workflow, not just a tool they open occasionally.

→ Low adoption = shallow embedding = higher churn risk. Every bug logged outside BrowserStack is a reason for an enterprise account to question renewal.

SCOPING ASSUMPTIONS

What we ruled out first

Not a Segment Issue

Low across Live & App Live, all regions, all product versions, new and existing customers.

Not a Regression

The feature never gained traction from day one - a cold start problem, not a sudden drop.

No External Trigger

No JIRA updates, competitor launches, or market events explain the gap.

Core question: Where in the journey from 'feature exists' to 'feature is used daily' are users dropping off — and what do we fix first?

FRAMEWORK

Breaking Down Adoption Into Measurable Stages

Before jumping to solutions, I decomposed adoption into three distinct stages - each with its own diagnostic question and success metric.

01

Conversion

Did they discover & set it up?

→ % who discovered the integration

→ % who initiated setup

→ % who completed setup

02

Activation

Did they use it for the first time?

→ % who attempted bug reporting

→ % who logged ≥1 bug successfully

→ Time-to-first-bug (TTV)

03

Usage

Did they find value & return?

→ Avg bugs logged / user / week

→ DAU / WAU / MAU growth

→ % users who logged >1 bug

Each stage narrows the funnel - we need to diagnose where users drop off before choosing a fix.

USERS

Who Isn't Adopting - and Why

Research sources: primary interviews with QA professionals, direct product exploration, and peer network discussions.

Primary Persona: QA Engineer

Methodical · Communicative · Detail-oriented

- ✕ Inconsistent setup: OAuth vs API Token flows create confusion
- ✕ Custom fields ignored — org-specific fields don't carry over
- ✕ No way to attach additional screenshots or session videos
- ✕ No two-way sync — comments and updates don't reflect back
- ✕ Can't save a bug draft to batch-report later
- ✕ No view of previously reported bugs from within the tool

Secondary Persona: Developer

Analytical · Time-conscious · Precision-driven

Developers rarely log bugs directly - but they receive, triage, and resolve them. Their concern is ticket quality: are the steps reproducible? Is the context sufficient to act on without back-and-forth?

Key insight: Developers value well-structured tickets with 'Steps to Reproduce' over speed of filing. Poor ticket quality is a shared pain point across both personas.

PRIORITISATION

Four Solutions. Three to Ship Now.

Scored by impact on the adoption funnel, implementation effort, and research confidence.

Solution	Impact	Effort	Priority	Funnel Stage	Key Metric
1. Fix setup flow inconsistency (OAuth everywhere)	**High**	Low	P1	Conversion	Setup completion rate
2. Allow additional file attachments in bug report	Med	Low	P2	Activation → Usage	Avg bugs logged / user
3. Auto-generate 'Steps to Reproduce' from session	**High**	Med	P3	Activation → Usage	Avg time per bug ↓, bugs/user ↑
4. Bug tracking & draft management hub	TBD	High	Hold	Usage	Needs further research

SOLUTION DESIGN

The Two Features That Move the Needle

Feature 2: Additional Attachments

Pain point addressed

Users can annotate and record sessions, but have no way to attach those files to the bug ticket they are filing.

User journey

1. Tap attachment icon in the bug panel
2. Browse and select files (system, Drive, iCloud)
3. Attachments added to existing list, count uncapped initially
4. Feasible: JIRA REST API supports multiple attachments natively

Feature 3: Auto-Generated Steps to Reproduce

Pain point addressed

Writing reproducible steps is time-consuming. Developers receiving tickets often lack enough context to act without follow-up.

User journey

1. Tap 'Track Steps' icon to start session recording
2. Each action (tap, scroll, pinch) is auto-captured with context
3. On 'Report Bug', attach the auto-generated steps document
4. Review & edit window: headers editable, steps removable

Competitive signal: Leading competitors already offer session-linked steps and multi-attachment support - parity is the floor, not the ceiling.

MEASURING SUCCESS

Ship Smart. Measure Everything.

Each feature launches with an A/B test and a defined success metric.

A/B Testing Approach

Roll out each feature to a defined % of integrated users. Compare adoption metrics between test and control groups before a full release. Gate progression on statistically significant improvement.

Stage 1

Conversion

Setup completion rate ↑

Stage 2

Activation

% users logging ≥1 bug ↑

Stage 3

Usage

Avg bugs/user/week ↑

Per feature

Feature-level

% bugs with attachments / STR

Hypothesis-driven · Instrumented from day one · Iterated on data